



Haedal - Volatile Vault V2

Security Assessment

CertiK Assessed on Dec 22nd, 2025





Certik Assessed on Dec 22nd, 2025

Haedal - Volatile Vault V2

The security assessment was prepared by Certik.

Executive Summary

TYPES

Staking

ECOSYSTEM

Sui (SUI)

METHODS

Manual Review, Static Analysis

LANGUAGE

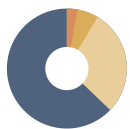
Move

TIMELINE

Preliminary comments published on 11/19/2025

Final report published on 12/22/2025

Vulnerability Summary



35

Total Findings

30

Resolved

1

Partially Resolved

4

Acknowledged

0

Declined

1 Centralization

1 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

2 Medium

2 Resolved



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

10 Minor

9 Resolved, 1 Partially Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

22 Informational

19 Resolved, 3 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | HAEDAL - VOLATILE VAULT V2

Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

[Overview](#)

[External Dependencies](#)

[Privileged Roles](#)

[Upgradeability](#)

Findings

[HVV-08 : Centralization Related Risks And Upgradability](#)

[HVV-02 : Inconsistent AdminCap Type Between `config.move` And `versioned.move`](#)

[HVV-31 : AUM Double-Counts Collected Fees/Rewards Under Certain Call Orders](#)

[HVV-03 : Missing Binding Between Close Cert And Vault May Allow Cross-Vault Reward Injection](#)

[HVV-11 : Potential Unsafe Removal Of Certified DLMM Pools Causing AUM Calculation Deadlock](#)

[HVV-14 : `open\\_position\\_curve\(\)`, `open\\_position\\_bid\\_ask\(\)`, And `open\\_position\\_spot\(\)` Do Not Check Whether The Corresponding Pool Is Listed In `market.certified\\_pools`](#)

[HVV-25 : Rounding Bias May Favor Users](#)

[HVV-26 : Potential Protocol Fee Bypass During Fee And Reward Collecting](#)

[HVV-32 : Potential Zero-Fee/Zero-Reward Micro-Harvesting \(Protocol Fee Bypass\)](#)

[HVV-33 : Potential Overflow In Pool Price Computation Used By Price Deviation Check Can Abort Transactions At Extreme Prices](#)

[HVV-35 : Incompatible With 0-Liquidity CLMM Position](#)

[HVV-36 : Potential Price Manipulation To `calculate\\_position\\_amounts`](#)

[HVV-37 : Inconsistent Price Type For `get\\_amount\\_by\\_liquidity`](#)

[HVV-01 : AUM Excluding Unpriced Assets Enabling LP Dilution](#)

[HVV-04 : Missing Check On `usd\\_price\\_age`](#)

[HVV-05 : Incomplete Logic For `remove\\_full\\_range\\_liquidity\\_by\\_percent`](#)

[HVV-07 : Inconsistency Between Code And Comment](#)

[HVV-09 : Potential Multiple Market Support In Vault](#)

[HVV-10 : Potential Inconsistent Pool Price Deviation Checks Across DLMM Adding Liquidity/Opening Position](#)

[HVV-12 : Remove On-Chain Dependency On Pyth `update\\_single\\_price\\_feed` To Prevent Unintended Abort](#)

[HVV-13 : Debug Import Left In Production Code](#)

[HVV-15 : Oracle Price Cache Not Cleared When Oracle Info Is Removed](#)

[HVV-16 : Incorrect Price Decimals Calculation In `calculate\\_oracle\\_prices\(\)`](#)

[HVV-17 : `flash\\_loan` Allows Identical `CoinTypeA` And `CoinTypeB`](#)

[HVV-18 : DLMM Fee/Reward Helpers Lack WithdrawCert-Vault Binding And State Checks](#)

[HVV-19 : Pyth Oracle Integration Ignores Confidence `conf` On Prices](#)

[HVV-20 : Potential Incorrect Slippage Application For Repay Amount](#)

[HVV-22 : Potential Issue Of Unfair Reward Distribution](#)

[HVV-24 : Version Control Did Not Emit Event](#)

[HVV-27 : Concerns About `update\\_price` And `get\\_price`](#)

[HVV-28 : Compilation Error In Unit Testing](#)

[HVV-29 : Event Amounts Exclude Fee Balances Credited To The Vault](#)

[HVV-30 : Keeper Operations Not Pausable](#)

[HVV-34 : `ClmmAddLiquidityEvent` Logs Allowed Amounts Instead Of Actual Used Amounts](#)

[HVV-38 : Potential DoS At Minimum Representable Price For `price == 1`](#)

Optimizations

[HVV-06 : Variable Naming](#)

[HVV-21 : The Complex Withdraw and Deposit Process Consumes Significant Gas](#)

Appendix

Disclaimer

CODEBASE | HAEDAL - VOLATILE VAULT V2

Repository

<https://github.com/haedallsd/haevault/tree/6038f593ec934b66f96b383b02da2fdc30bd328a>

Commit

[6038f593ec934b66f96b383b02da2fdc30bd328a](#)

[5b6b28f4b14de4f807e6efd6d84381ff8311ac9b](#)

[9b220591ce962753e0dedd4defb897f66fc839a8](#)

[616b3fdbdf3a5ef8dbf227f4d6f8ceb46d6bfc70](#)

[6d94df586ccad1cad3f2f6bd7190962eab4f4873](#)

[d24e26b8f36e50864daf0db36b8bc409f1c33cbf](#)

[467e91a4a818f72f8332ea8832ecce72edc4a3b4](#)

[f9881cdd89b22098509c2d34533688df76d4ee9b](#)

[8bedfc9b0660091edc43cd4b489abcf28b364c9a](#)

[8169f247cdb761eefe12479ecffabe4abf547af3](#)

[f5b6636a99ae42cff9f912cf4c7512ef0239a476](#)

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | HAEDAL - VOLATILE VAULT V2

This audit was conducted for Haedal to evaluate the security and correctness of the smart contracts associated with the Haedal - Volatile Vault V2 project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | HAEDAL - VOLATILE VAULT V2

Overview

The **Haedal - Volatile Vault V2** is a volatile-asset liquidity vault that mints LP shares, handles deposits and withdrawals, tracks AUM, and manages concentrated liquidity positions on Cetus DLMM. It uses Pyth oracle prices for valuation and checks, includes access control, global config, versioning, and a multi-asset balance bag, and supports rebalancing plus fee and reward collection under keeper/admin controls.

External Dependencies

The project is developed using the Move language and running on the top of the Sui blockchain. The vulnerability and the updates of the language/Sui framework may affect the project as a whole. As the Sui network is rapidly evolving, to avoid any potential compatibility issues and take advantage of new features and improvements, the client should upgrade the Sui framework to the most recent version. Additionally, staying informed about any upcoming updates or changes to the language or framework can help ensure the project remains secure and compatible.

Dependency of the **Haedal - Volatile Vault V2**:

```
Pyth = {git = "https://github.com/pyth-network/pyth-crosschain.git", subdir =
"target_chains/sui/contracts", rev = "main"}
# CetusDlmm = { git = "https://github.com/CetusProtocol/cetus-dlmm-interface.git",
subdir = "packages/dlmm", rev = "1e1c0171d3560453e2b012c40ec8f5c1fc46d484", override
= true }
CetusDlmm = {local = "../../../../../github_workspace/dlmm/packages/cetus_dlmm"}
DlmmRouter = { local = "../../../../../github_workspace/dlmm/packages/dlmm_router" }
IntegerMate = { git = "https://github.com/CetusProtocol/integer-mate.git", rev =
"mainnet-v1.3.0", override = true }
```

The above dependencies are not within the current audit scope and serve as a black box. Modules/Contracts within the module are assumed to be valid and non-vulnerable actors in this audit and implement proper logic to collaborate with the current project and other modules.

Privileged Roles

To set up the project correctly and ensure that the project functions properly, owners of the following objects are able to use privileged functions, more details in **GLOBAL-01: Centralization Related Risks And Upgradability**.

The advantage of the privileged role in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to serve the community best. It is also worthy of note the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the key pairs of privileged accounts are compromised, the project could have devastating consequences.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Furthermore, any plan to invoke the aforementioned functions should also be considered to move to the execution queue of the `Timelock` contract.

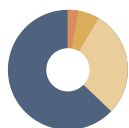
I Upgradeability

Developers working with the Sui blockchain have the ability to upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and publisher's key store should be handled with caution to prevent any unexpected loss. Additionally, it is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

Reference:

- [Sui Package Upgrades](#)
- [Custom Policies](#)

FINDINGS | HAEDAL - VOLATILE VAULT V2



35

Total Findings

0

Critical

1

Centralization

0

Major

2

Medium

10

Minor

22

Informational

This report has been prepared for Haedal to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 35 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
HVV-08	Centralization Related Risks And Upgradability	Centralization	Centralization	● Acknowledged
HVV-02	Inconsistent AdminCap Type Between <code>config.move</code> And <code>versioned.move</code>	Inconsistency	Medium	● Resolved
HVV-31	AUM Double-Counts Collected Fees/Rewards Under Certain Call Orders	Design Issue	Medium	● Resolved
HVV-03	Missing Binding Between Close Cert And Vault May Allow Cross-Vault Reward Injection	Logical Issue	Minor	● Resolved
HVV-11	Potential Unsafe Removal Of Certified DLMM Pools Causing AUM Calculation Deadlock	Design Issue	Minor	● Resolved
HVV-14	<code>open_position_curve()</code> , <code>open_position_bid_ask()</code> , And <code>open_position_spot()</code> Do Not Check Whether The Corresponding Pool Is Listed In <code>market.certified_pools</code>	Logical Issue	Minor	● Resolved
HVV-25	Rounding Bias May Favor Users	Volatile Code	Minor	● Resolved
HVV-26	Potential Protocol Fee Bypass During Fee And Reward Collecting	Volatile Code	Minor	● Resolved

ID	Title	Category	Severity	Status
HVV-32	Potential Zero-Fee/Zero-Reward Micro-Harvesting (Protocol Fee Bypass)	Design Issue	Minor	● Resolved
HVV-33	Potential Overflow In Pool Price Computation Used By Price Deviation Check Can Abort Transactions At Extreme Prices	Volatile Code	Minor	● Resolved
HVV-35	Incompatible With 0-Liquidity CLMM Position	Volatile Code	Minor	● Resolved
HVV-36	Potential Price Manipulation To <code>calculate_position_amounts</code>	Design Issue	Minor	● Partially Resolved
HVV-37	Inconsistent Price Type For <code>get_amount_by_liquidity</code>	Volatile Code	Minor	● Resolved
HVV-01	AUM Excluding Unpriced Assets Enabling LP Dilution	Volatile Code	Informational	● Acknowledged
HVV-04	Missing Check On <code>usd_price_age</code>	Volatile Code	Informational	● Resolved
HVV-05	Incomplete Logic For <code>remove_full_range_liquidity_by_per_cent</code>	Coding Issue	Informational	● Resolved
HVV-07	Inconsistency Between Code And Comment	Coding Issue	Informational	● Resolved
HVV-09	Potential Multiple Market Support In Vault	Design Issue	Informational	● Acknowledged
HVV-10	Potential Inconsistent Pool Price Deviation Checks Across DLMM Adding Liquidity/Opening Position	Design Issue	Informational	● Resolved
HVV-12	Remove On-Chain Dependency On Pyth <code>update_single_price_feed</code> To Prevent Unintended Abort	Design Issue, Code Optimization	Informational	● Resolved
HVV-13	Debug Import Left In Production Code	Coding Style	Informational	● Resolved

ID	Title	Category	Severity	Status
HVV-15	Oracle Price Cache Not Cleared When Oracle Info Is Removed	Volatile Code	Informational	● Resolved
HVV-16	Incorrect Price Decimals Calculation In <code>calculate_oracle_prices()</code>	Incorrect Calculation	Informational	● Resolved
HVV-17	<code>flash_loan</code> Allows Identical <code>CoinTypeA</code> And <code>CoinTypeB</code>	Volatile Code	Informational	● Resolved
HVV-18	DLMM Fee/Reward Helpers Lack WithdrawCert-Vault Binding And State Checks	Volatile Code	Informational	● Resolved
HVV-19	Pyth Oracle Integration Ignores Confidence <code>conf</code> On Prices	Volatile Code	Informational	● Resolved
HVV-20	Potential Incorrect Slippage Application For Repay Amount	Design Issue	Informational	● Resolved
HVV-22	Potential Issue Of Unfair Reward Distribution	Design Issue	Informational	● Resolved
HVV-24	Version Control Did Not Emit Event	Logical Issue	Informational	● Resolved
HVV-27	Concerns About <code>update_price</code> And <code>get_price</code>	Volatile Code	Informational	● Resolved
HVV-28	Compilation Error In Unit Testing	Coding Issue	Informational	● Resolved
HVV-29	Event Amounts Exclude Fee Balances Credited To The Vault	Volatile Code	Informational	● Resolved
HVV-30	Keeper Operations Not Pausable	Volatile Code	Informational	● Acknowledged
HVV-34	<code>ClimmAddLiquidityEvent</code> Logs Allowed Amounts Instead Of Actual Used Amounts	Volatile Code	Informational	● Resolved
HVV-38	Potential DoS At Minimum Representable Price For <code>price == 1</code>	Volatile Code	Informational	● Resolved

HVV-08 | Centralization Related Risks And Upgradability

Category	Severity	Location	Status
Centralization	● Centralization		● Acknowledged

Description

The role **AdminCap** has authority over the functions:

- config::set_roles
- config::remove_member
- config::emergency_unpause
- versioned::upgrade_v2

The role **ACL_CLAIM_PROTOCOL_FEE** has authority over the functions:

- vault::claim_protocol_fee

The role **ACL_KEEPER_MANAGER** has authority over the functions:

- vault::open_dlmm_position
- vault::open_position_bid_ask
- vault::open_position_spot
- vault::open_position_curve
- vault::new_dlmm_add_liquidity_cert
- vault::add_dlmm_liquidity_on_bin
- vault::repay_dlmm_add_liquidity
- vault::add_dlmm_liquidity
- vault::add_dlmm_liquidity_spot
- vault::add_dlmm_liquidity_curve
- vault::add_liquidity_bid_ask
- vault::remove_dlmm_liquidity_by_percent
- vault::close_dlmm_position_v2
- vault::flash_loan
- vault::repay_flash_loan

The role **ACL_VAULT_MANAGER** has authority over the functions:

- vault::create_vault
- vault::add_dlmm_market
- vault::add_dlmm_certified_pool

- vault::remove_dlm certified_pool
- vault::update_dlm certified_pool
- vault::update_active_status
- vault::update_hard_cap
- vault::update_protocol_fee_rate

The role **ACL_ORACLE_MANAGER** has authority over the functions:

- pyth_oracle::add_oracle_info
- pyth_oracle::add_oracle_info_v2
- pyth_oracle::remove_oracle_info
- pyth_oracle::update_price_age

The role **ACL_EMERGENCY_PAUSE** has authority over the functions:

- config::emergency_pause

The role **ACL_CONFIG_MANAGER** has authority over the functions:

- config::update_protocol_fee_rate
- config::set_swap_slippage
- config::set_aum_safety_margin_bps

In addition, developers working with the Sui blockchain can upgrade packages based on their software iteration requirements. However, this also means that the `UpgradeCap` and deployer's key store should be handled carefully to prevent any unexpected losses. It is important to inform the community about any upgrade plans to address concerns related to centralization and ensure transparency.

More information can be found:

- [Sui Package Upgrades](#)
- [Third-Party Package upgrades](#)

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

I Alleviation

[Haedal, 11/20/2025]: We acknowledge the centralization concerns. To mitigate these risks, the protocol's critical permissions are structured to avoid reliance on a single operator. Both the **AdminCap and UpgradeCap will be controlled by a multi-signature address**, so any administrative configuration or upgrade action requires approval from multiple signers rather than a single key holder.

Additionally, all **ACL-restricted roles are managed through hardware wallets**, ensuring that privileged operations-such as vault management, keeper actions, oracle updates, and protocol fee claims-are executed using securely stored keys with strong resistance to compromise.

These controls meaningfully reduce single-point-of-failure risks while maintaining the operational flexibility required for protocol maintenance and upgrades.

[CertiK, 11/20/2025]: CertiK strongly encourages the project team to periodically revisit the private-key security management for all addresses associated with centralized roles.

HVV-02 Inconsistent AdminCap Type Between `config.move` And `versioned.move`

Category	Severity	Location	Status
Inconsistency	● Medium	packages/volatile_vault_v2/sources/config.move (vault-v2): 9	● Resolved

Description

`versioned.move` uses `cetusdlmm::admin_cap::AdminCap` while `config.move` uses `volatile_vault_v2::admin_cap::AdminCap`. This splits administrative authority across two different modules, preventing the `upgrade` function from being executed.

`version.move`:

```
use cetusdlmm::admin_cap::AdminCap;

public fun upgrade(versioned: &mut Versioned, _: &AdminCap) {
    assert!(versioned.version < VERSION, EInvalidVersion);
    versioned.version = VERSION;
}
```

`config.move`:

```
use volatile_vault_v2::admin_cap::AdminCap;
```

Recommendation

Using one AdminCap type across all modules.

Alleviation

[Haedal, 11/18/2025]: The team heeded the advice and resolved the issue by using

`volatile_vault_v2::admin_cap::AdminCap` in commit [5b6b28f4b14de4f807e6efd6d84381ff8311ac9b](#).

HVV-31 | AUM Double-Counts Collected Fees/Rewards Under Certain Call Orders

Category	Severity	Location	Status
Design Issue	● Medium	packages/volatile_vault_v2/sources/vault.move (12/09 - 467e91): 876	● Resolved

Description

The `volatile_vault_v2::vault` AUM calculation is sensitive to the order in which certain public fun are invoked within a single programmable transaction block (PTB).

If a caller first runs `calculate_clmm_position_amounts` / `calculate_dlmm_position_amounts`, then calls `collect_clmm_fee` / `collect_dlmm_fee` or `collect_*_reward`, and finally calls `calculate_aum`, the same fee/reward amounts are counted both in the position totals and in `vault.buffer_assets`.

This breaks the invariant “AUM = on-chain asset value” and can lead to inflated `last_aum`, even though it does not currently cause a direct profit vector for users.

A user who accidentally misorders calls in a PTB (e.g. for `deposit`) can harm themselves by minting fewer LP tokens than intended due to an inflated AUM denominator.

Recommendation

Recommend implementing stronger limitations to `calculate_aum` and ensure there is no fee collection and reward collection with the current tx digest.

Alleviation

[Haedal, 12/11/2025]: The team heeded the advice and resolved the issue by adding validation when `calculate_aum` in commit [f9881cdd89b22098509c2d34533688df76d4ee9b](#).

HVV-03 | Missing Binding Between Close Cert And Vault May Allow Cross-Vault Reward Injection

Category	Severity	Location	Status
Logical Issue	Minor	packages/volatile_vault_v2/sources/vault.move (vault-v2): 1758	Resolved

Description

`market_dlm::close_position` issues `DLMMPositionCloseCert` containing only `position_id` (no `vault_id` / `market_id`). `vault.collect_dlm_reward_from_close_cert` does not verify that this `position_id` (or `cert`) belongs to the vault's DLMMMarket. As a result, rewards from other positions can be claimed into this vault.

Recommendation

Bind `DLMMPositionCloseCert` to the issuing vault/market at creation time, and verify the relationship when collecting the reward.

Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by adding validation on market id in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#).

HVV-11 | Potential Unsafe Removal Of Certified DLMM Pools Causing AUM Calculation Deadlock

Category	Severity	Location	Status
Design Issue	● Minor	packages/volatile_vault_v2/sources/vault.move (11/17 - 5b6b28): 541	● Resolved

Description

After a pool is removed from `certified_pools`, the AUM calculation flow may fail during the per-position valuation step. The valuation path checks price deviation using the pool's certified deviation parameter retrieved via `certified_pools.get(&object::id(pool))`. Once the pool is removed, this `.get` access aborts, so position amounts are never refreshed, `last_amounts` cannot be validated, and AUM cannot be calculated in that transaction. As a result, the deposit operation could encounter denial-of-service issue.

```

public(package) fun last_amounts(market: &DLMMMarket, ctx: &TxContext):
VecMap<TypeName, u64> {
    ...
    while (idx < market.position_keys.length()) {
        let position_id = *market.position_keys.keys().borrow(idx);
    @> let position = market.positions.borrow<ID, DLMMPosition>(position_id);
        ...
    }

public(package) fun calculate_position_amounts<CoinTypeA, CoinTypeB>(
    market: &mut DLMMMarket,
    pool: &mut Pool<CoinTypeA, CoinTypeB>,
    position_id: ID,
    versioned: &cetusdlmm::versioned::Versioned,
    clk: &Clock,
    ctx: &TxContext,
) {
    ...
    @> let wrapped_position = market.positions.borrow_mut<ID, DLMMPosition>
(position_id);
    ...
    wrapped_position.amounts = amounts;
    ...
}

public fun calculate_dlmm_position_amounts<CoinTypeA, CoinTypeB, T>(
    vault: &mut Vault<T>,
    pyth_oracle: &PythOracle,
    dlmm_pool: &mut DlmmPool<CoinTypeA, CoinTypeB>,
    position_id: ID,
    dlmm_versioned: &DLMMVersioned,
    versioned: &Versioned,
    clk: &Clock,
    ctx: &TxContext,
) {
    ...
    @> dlmm_market.check_pool_price_deviation(dlmm_pool, pyth_oracle, clk);

    market_dlmm::calculate_position_amounts(
        dlmm_market,
        dlmm_pool,
        position_id,
        dlmm_versioned,
        clk,
        ctx,
    );
}

public fun check_pool_price_deviation<CoinTypeA, CoinTypeB>(

```

```
market: &DLMMMarket,  
pool: &Pool<CoinTypeA, CoinTypeB>,  
pyth_oracle: &PythOracle,  
clk: &Clock,  
) {  
    ...  
@> let derivation = *market.certified_pools.get(&object::id(pool));  
    ...  
}
```

Additionally, after a pool is removed from `certified_pools`, add-liquidity paths still allow adding liquidity to positions in that pool, since several entry points do not require the pool to remain certified before proceeding.

Recommendation

Recommend that the team double-check the intended design or verify that there are no active positions for the pool before removing it from certified pools. Also, we would like to check with the team about the purpose for

`check_pool_price_deviation` during calculating AUM.

Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by adding position check of

`market_dlmm::remove_certified_pool()` in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#).

HVV-14 `open_position_curve()` , `open_position_bid_ask()` , And `open_position_spot()` Do Not Check Whether The Corresponding Pool Is Listed In `market.certified_pools`

Category	Severity	Location	Status
Logical Issue	Minor	packages/volatile_vault_v2/sources/vault.move (vault-v2): 1206, 1271, 1336	Resolved

Description

The functions `open_position_curve` , `open_position_bid_ask` , and `open_position_spot` do not check whether the corresponding pool is included in `market.certified_pools` when opening a position.

However, `dlmm_market.new_position()` inside `open_dlmm_position()` does include the relevant verification.

`vault.move`

```

1079     let dlmm_market = vault
1080         .markets
1081         .borrow_mut<vector<u8>, market_dlmm::DLMMMarket>(DLMM_MARKET_KEY);
1082 @> dlmm_market.new_position(position, ctx);

```

`market_dlmm.move`

```

129 public(package) fun new_position(market: &mut DLMMMarket, position: Position,
ctx: &mut TxContext) {
130 @> assert!(market.certified_pools.contains(&position.pool_id()),
EUncertifiedPool);
131     let position_id = object::id(&position);
132     let wrap_position = DLMMPosition {
133         id: object::new(ctx),
134         position,
135         amounts: vec_map::empty(),
136         last_txs: vec_map::empty(),
137     };
138     market.position_keys.insert(position_id);
139     market.positions.add(position_id, wrap_position);
140 }

```

Recommendation

We recommend adding a pool check of `market.certified_pools` , similar to what is done in `open_dlmm_position()` .

I Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by adding pool check of `market.certified_pools` in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#)

HVV-25 | Rounding Bias May Favor Users

Category	Severity	Location	Status
Volatile Code	Minor	packages/volatile_vault_v2/sources/vault.move (vault-v2): 2050, 2067	Resolved

Description

The following concerns are related to the rounding in the current implementation, which may allow :

Flash Loan Repayment Rounding-down

`repay_amount` `flash_loan` is calculated using `mul_div_floor`, as a result, if `(price * loan) < decimals` the result is 0, and the protocol would suffer a loss.

LP Share Insolvency

The function `get_user_share_by_lp_amount` uses `mul_div_round` (nearest) to calculate user withdrawals, which may result in over-withdrawal when rounding-up.

Deviation Check Bypass

The function `check_pool_price_deviation` uses `mul_div_floor` to calculate the percentage difference, which allows the downgrade of the difference due to rounding down.

In "Favor the Protocol" principle:

- Debt/Repayment: Always round UP (users pay slightly more or exact).
- Withdrawal/Claims: Always round DOWN (users receive slightly less or exact).
- Safety Checks: Always round UP on the "bad" metric (deviation) to trigger alerts sooner.

Recommendation

We would like to double-check with the team about the rounding issue above.

Alleviation

[Haedal, 11/26/2025]: <https://github.com/haedallsd/haevault/pull/8/commits/ae1ef9649e7beee1f61e4bd61ff95f7adf6d8dbf>

[CertiK, 11/26/2025]:

Recommend using the ceiling for the calculation of `repay_amount`.

```
let repay_amount = full_math_u128::mul_div_round(
    base_price,
    loan_amount as u128,
    std::u128::pow(10, relative_price_multiplier_decimal!()),
);
```

Recommend using the ceiling for the calculation of `price_diff` .

```
let price_diff = full_math_u128::mul_div_round(
    BASIS_POINTS_DIVISOR as u128,
    diff,
    pool_oracle_price,
);
```

[Haedal, 12/10/2025]: The team heeded the advice and resolved the issue by updating the rounding direction in commit [0cd6eb12225ebef856e69a27e43e0da1bb216482](https://github.com/haedal/haedal-vault-v2/commit/0cd6eb12225ebef856e69a27e43e0da1bb216482).

HVV-26 | Potential Protocol Fee Bypass During Fee And Reward Collecting

Category	Severity	Location	Status
Volatile Code	Minor	packages/volatile_vault_v2/sources/market_dlmm.move (vault-v2): 732~750; packages/volatile_vault_v2/sources/vault.move (11/21 - 9b2205): 1006~1009, 1071, 1965~1967, 1996	Resolved

Description

When collecting fees and reward from an open position, the protocol will invoke `merge_protocol_asset` to handle protocol fee:

```
1006 let protocol_fee_a = vault.merge_protocol_asset(&mut balance_a);
1007 let protocol_fee_b = vault.merge_protocol_asset(&mut balance_b);
1008 vault.buffer_assets.join(balance_a);
1009 vault.buffer_assets.join(balance_b);
```

```
1071 let protocol_fee = vault.merge_protocol_asset(&mut reward);
1072 vault.buffer_assets.join(reward);
```

However, when `close_dlmm_position_v2`, the entire principal plus accrued fees and reward is returned, and the code skips the fee merge:

```
1965 vault.buffer_assets.join(balance_a);
1966 vault.buffer_assets.join(balance_b);
```

`collect_dlmm_reward_from_close_cert_v2` also forget to handle protocol fee of reward:

```
1996 vault.buffer_assets.join(reward_balance);
```

Similarly, in `calculate_dlmm_position_amounts`, which needs to be called before a deposit, this part is also not included in the protocol fee calculation:

```
732 let mut amounts: VecMap<TypeName, u64> = vec_map::empty();
733 let coin_type_a = type_name::with_defining_ids<CoinTypeA>();
734 let coin_type_b = type_name::with_defining_ids<CoinTypeB>();
735 let (amount_a, amount_b) = detail.position_detail_amounts();
736 let (fee_a, fee_b) = detail.position_detail_fees();
737 amounts.insert(coin_type_a, amount_a + fee_a);
738 amounts.insert(coin_type_b, amount_b + fee_b);
739 let rewards = detail.position_detail_rewards();
740 let mut idx = 0;
741 while (idx < rewards.length()) {
742     let (reward_type, reward_amount) = rewards.get_entry_by_idx(idx);
743     if (amounts.contains(reward_type)) {
744         let amount = amounts.get_mut(reward_type);
745         *amount = *amount + *reward_amount;
746     } else {
747         amounts.insert(*reward_type, *reward_amount);
748     };
749     idx = idx + 1;
750 };
```

Recommendation

We recommend adding the corresponding fee-charging mechanism.

Alleviation

[Haedal, 11/26/2025]: <https://github.com/haedallsd/haevault/pull/8/commits/6d94df586ccad1cad3f2f6bd7190962eab4f4873>

[Certik, 11/26/2025]: We want to remind the team that the `calculate_dlmm_position_amounts` function does not account for the protocol fee, which could lead to an overestimate of the protocol's AUM and result in the user receiving fewer LP tokens than expected.

[Haedal, 12/10/2025]: The team heeded the advice and resolved the issue by calculating the protocol fee in the `calculate_dlmm_position_amounts` in commit [0cd6eb12225ebef856e69a27e43e0da1bb216482](https://github.com/haedallsd/haevault/commit/0cd6eb12225ebef856e69a27e43e0da1bb216482).

HVV-32 Potential Zero-Fee/Zero-Reward Micro-Harvesting (Protocol Fee Bypass)

Category	Severity	Location	Status
Design Issue	● Minor		● Resolved

Description

`collect_clmm_fee` is publicly callable and does not enforce any role, minimum amount, or call-frequency constraint. It calls `merge_protocol_asset` on the harvested balances, which computes the protocol cut as `fee_amount = floor(balance.value() * protocol_fee_rate / protocol_fee_denominator)`.

Because `fee_amount` is calculated with floor on the amount harvested in that single call, if the harvested amount is small enough, `fee_amount` becomes 0. The function still transfers the entire harvested balance into `vault.buffer_assets`, but the `protocol_fees` bag receives nothing.

Since `collect_clmm_fee` can be called arbitrarily often by anyone, an external caller can repeatedly harvest very small amounts (before fees accrue to a size that would produce a non-zero protocol fee), effectively bypassing the protocol-fee mechanism.

Similar issues exist in `collect_clmm_reward`, `collect_dlmm_fee`, and `collect_dlmm_reward`.

Recommendation

Recommend considering the ceiling protocol fee calculation.

Alleviation

[Haedal, 12/12/2025]: The team heeded the advice and resolved the issue by using ceiling in commit [8bedfc9b0660091edc43cd4b489abcf28b364c9a](#).

[CertiK, 12/12/2025]: It is worth noting that if the method is frequently invoked, the fee may be fully converted to the protocol fee.

HVV-33 Potential Overflow In Pool Price Computation Used By Price Deviation Check Can Abort Transactions At Extreme Prices

Category	Severity	Location	Status
Volatile Code	Minor	packages/volatile_vault_v2/sources/market_clmm.move (12/09 - 467e91): 491-502	Resolved

Description

`check_pool_price_deviation` relies on `get_current_pool_price` to compare the CLMM pool price to the oracle price. `get_current_pool_price` computes price as: $\text{price} = (\text{sqrt_price} * 10^{\text{REL}} / 2^{64})^2 / 10^{\text{REL}}$ using u256 intermediates (REL = relative_price_multiplier_decimal = 20). For pools with very high current sqrt prices (near tick/extreme bounds), the overflow may be generated when converting a u256 value to a u128:

```
497     let price =
498         (
499             std::u256::pow(sqrt_price_after_rs, 2) / std::u256::pow(10,
relative_price_multiplier_decimal!),
500         ) as u128;
```

A similar issue applies to DLMM, for example, when `bin_step` is 1 and `active_id` is 443636.

```
874     let price = price / (1u256 << 64);
875     price as u128
876
```

Proof of Concept

We can reuse the following test to verify the issue:

```
#[test]
fun test_check_pool_price_extreme() {
    let mut ctx = tx_context::dummy();
    let clk = sui::clock::create_for_testing(&mut ctx);
    let mut pyth_oracle = pyth_oracle::new_pyth_oracle_for_test(&mut ctx);
    pyth_oracle.update_price_for_test<SUI>(16100000000, 9, &clk); // 24179058200
    pyth_oracle.update_price_for_test<USDC>(10000000000, 6, &clk); // 9998490900

    let (admin_cap, clmm_config) = config::new_global_config_for_test(&mut ctx,
2000);
    let clmm_pool = pool::new_for_test<USDC, SUI>(
        10,
        79226673515401279992447579055, // @audit MAX_SQRT_PRICE_X64
        5000,
        std::string::utf8(b"https://v.com"),
        1,
        &clk,
        &mut ctx,
    );
    let base_price = pyth_oracle.get_price<USDC>(&clk);
    let quote_price = pyth_oracle.get_price<SUI>(&clk);
    std::debug::print(&base_price);
    std::debug::print(&quote_price);
    let (pool_oracle_price, _) = calculate_prices_v2(&base_price, &quote_price);
    std::debug::print(&pool_oracle_price);

    let pool_current_price = get_current_pool_price(&clmm_pool);
    std::debug::print(&pool_current_price);

    // Validate price deviation of oracle_price and pool current price
    let diff = std::u128::diff(pool_oracle_price, pool_current_price);
    let derivation = 200;
    let price_diff = full_math_u128::mul_div_floor(
        10000,
        diff,
        pool_oracle_price,
    );
    std::debug::print(&price_diff);
    assert!(price_diff <= derivation, 0);

    sui::transfer::public_share_object(clmm_pool);
    sui::transfer::public_transfer(admin_cap, ctx.sender());
    transfer::public_share_object(clmm_config);

    pyth_oracle.destroy_pyth_oracle();

    clk.destroy_for_testing();
}
```

Output:

```
| error[E11001]: test failure
|   └─ ./sources/market_clmm.move:498:9
|
|   |
| 491 |   public fun get_current_pool_price<CoinTypeA, CoinTypeB>(
|   |   ----- In this function in
volatile_vault_v2::market_clmm
|   |
| 498 |   (
| 499 |   |   std::u256::pow(sqrt_price_after_rs, 2) / std::u256::pow(10,
relative_price_multiplier_decimal!),
| 500 |   |   ) as u128;
|   |   ^ Test was not expected to error, but it gave an
arithmetic error originating in the module volatile_vault_v2::market_clmm rooted
here
```

Recommendation

Recommend reconsidering the return value type for the `get_current_pool_price` with CLMM.

Alleviation

[Haedal, 12/11/2025]: The team heeded the advice and resolved the issue by updating

`relative_price_multiplier_decimal` in commit [f9881cdd89b22098509c2d34533688df76d4ee9b](https://github.com/certik/haedal/commit/f9881cdd89b22098509c2d34533688df76d4ee9b).

HVV-35 | Incompatible With 0-Liquidity CLMM Position

Category	Severity	Location	Status
Volatile Code	Minor	packages/volatile_vault_v2/sources/vault.move (12/12 - 8bedfc): 1529	Resolved

Description

`open_clmm_position` always opens a new CLMM position, then calls `clmm_market.increase_liquidity` but ignores its return values and does not validate that any liquidity was actually added.

In `market_clmm::increase_liquidity`, if the computed `delta_liquidity` is 0, it returns `(0, 0, 0)` early and leaves the position with zero liquidity. This can happen trivially when `(amount_a == 0 && amount_b == 0)`, and can also happen due to rounding with small amounts or certain tick ranges.

A zero-liquidity position becomes effectively stuck because Cetus CLMM `pool::remove_liquidity` enforces `delta_liquidity > 0`. Vault flows like `close_clmm_position` (which removes `position_info.info_liquidity()`, potentially 0) and `withdraw_clmm` (which computes `delta_liquidity` proportionally, potentially 0) will abort when attempting to remove liquidity from such a position.

```
public fun remove_liquidity<CoinTypeA, CoinTypeB>(
    config: &GlobalConfig,
    pool: &mut Pool<CoinTypeA, CoinTypeB>,
    position_nft: &mut Position,
    delta_liquidity: u128,
    clock: &Clock,
): (Balance<CoinTypeA>, Balance<CoinTypeB>) {
    checked_package_version(config);
    assert!(is_allow_remove_liquidity(pool), EOperationNotPermitted);
    assert!(delta_liquidity > 0, ELiquidityIsZero);
    ...
}
```

Since withdrawals require all positions to be withdrawn/closed before finalization, a single stuck zero-liquidity position can block withdraw completion for all users.

Recommendation

Recommend invoking `remove_liquidity` when `delta_liquidity > 0`.

Alleviation

[Haedal, 12/16/2025]: The team heeded the advice and resolved the issue by handling the condition when `delta_liquidity == 0` in commit [15bf884f249bafb70c76091dccdb14e8ab07c47e](#).

HVV-36 | Potential Price Manipulation To `calculate_position_amounts`

Category	Severity	Location	Status
Design Issue	Minor	packages/volatile_vault_v2/sources/market_clmm.move (12/12 - 8bedfc): 411; packages/volatile_vault_v2/sources/market_dlmm.move (12/12 - 8bedfc): 823	Partially Resolved

Description

The `calculate_position_amounts` logic in `market_clmm.move` and `market_dlmm.move` computes `amount0` and `amount1` based on the current pool price. While the `check_pool_price_deviation` safeguard in `vault.move` prevents large differences between the pool price and the oracle price, users may still manipulate the pool price (via swap or flashloan operations) within the allowed deviation. This could potentially allow them to gain an advantage, such as lowering AUM, to mint more LP tokens.

`market_clmm.move`:

```
let current_tick = pool.current_tick_index();
...
let (amount_a, amount_b) = clmm_math::get_amount_by_liquidity(
    tick_lower,
    tick_upper,
    @> current_tick,
    current_sqrt_price_val,
    position_info.info_liquidity(),
    false,
);
```

`market_dlmm.move`:

```
@> let detail = pool::refresh_position_info(pool, position_id, versioned, clk,
    ctx);

let (amount_a, amount_b) = detail.position_detail_amounts();
```

Recommendation

Consider calculating `amount0` & `amount1` based on the oracle price rather than the spot price used from pool.

Alleviation

[Haedal, 12/18/2025]: Thank you for your valuable feedback. We have made the requested changes regarding the oracle price calculation for CLMM and DLMM positions.

Please review the updated version . We appreciate your review.

<https://github.com/haedallsd/haevault/pull/8/commits/8169f247cdb761eefe12479ecffabe4abf547af3>

[CertiK, 12/19/2025]: The vault's DLMM AUM is produced by a multi-step fixed-point pipeline: oracle USD prices → a relative pool price (scaled) → a Q64.64 price → expected_active_id via a log-based mapping → per-bin pro-rata position amounts plus conditional A <-> B normalization around expected_active_id → protocol-fee haircut on fees/rewards → final AUM aggregation in USD/quote. Many steps involve floor/ceil/truncation and fixed-point quantization, so small rounding errors (including a potential one-bin shift in expected_active_id, as get_id_from_price will rounding to 0) can propagate and may push the final AUM either upward or downward, depending on the position's inventory near the boundary bins; there is no single rounding direction that deterministically "controls" the AUM bias.

Since last_aum is primarily used to price LP minting on deposits (shares minted are inversely proportional to AUM), a protocol-conservative approach is to avoid underestimating AUM at deposit time, which could otherwise dilute existing LPs via share-minting arbitrage. If tuning every rounding corner case is too complex, an alternative way could be applying an explicit, transparent safety margin for deposit pricing only (e.g., multiply AUM by $1+\epsilon$, with ϵ configurable), which effectively behaves like a small, controlled deposit fee.

If AUM is later reused in risk-sensitive modules (e.g., lending/collateral), the conservative direction typically flips: the protocol should prefer a pessimistic (haircut) valuation to reduce insolvency risk.

Given the complexity and sensitivity of this AUM calculation, we strongly recommend extensive scenario-based testing (including boundary-bin and rounding edge cases) before finalizing the design, and we'd be happy to discuss and iterate on any feedback or alternative approaches the team has in mind.

[Haedal, 12/22/2025]:

Hi, we add a aum_safety_margin_bps parameter to protect the protocol. pls help check

<https://github.com/haedallsd/haevault/pull/8/commits/f5b6636a99ae42cff9f912cf4c7512ef0239a476>

[CertiK, 12/22/2025]:

The team has introduced a new safety margin parameter designed to slightly increase the final AUM, functioning similarly to a deposit fee. This adjustment aims to mitigate the risks associated with the underestimation of AUM due to rounding errors and loss of precision.

The team must continue to monitor the potential accumulation of precision loss in AUM calculations as the number of positions increases. Accordingly, the team needs to adjust the appropriate margin parameters and the deviation threshold dynamically. Since these parameters are set off-chain and are not within the scope of auditing, we have marked this issue as Partially Resolved.

HVV-37 | Inconsistent Price Type For `get_amount_by_liquidity`

Category	Severity	Location	Status
Volatile Code	Minor	packages/volatile_vault_v2/sources/market_clmm.move (12/18 - 8169f2): 419	Resolved

Description

In `market_clmm::calculate_position_amounts`, the code correctly derives `oracle_sqrt_price = price_to_sqrt_price(pool_oracle_price)` and `oracle_price_tick = tick_math::get_tick_at_sqrt_price(oracle_sqrt_price)`. However, when calling `cetus_clmm::clmm_math::get_amount_by_liquidity`, it passes `pool_oracle_price` as the `current_sqrt_price` argument.

This is a type/units mismatch: `get_amount_by_liquidity(..., current_sqrt_price: u128, ...)` expects a Q64.64 sqrt price (like Cetus pool's `current_sqrt_price`), but `pool_oracle_price` is a linear price scaled by `relative_price_multiplier_decimal` (1e18). Feeding a linear price into formulas that assume sqrt-price fixed-point will compute materially incorrect `(amount_a, amount_b)` for positions.

Recommendation

Recommend using sqrt price instead.

Alleviation

[Haedal, 12/22/2025]: The team heeded the advice and resolved the issue by using the sqrt price in commit [30c92359c2fb557d7ef39d4375e77a0d5e389804](#).

HVV-01 | AUM Excluding Unpriced Assets Enabling LP Dilution

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/vault.move (vault-v2): 617	● Acknowledged

Description

Assets without oracle info are ignored by `calculate_aum` but later distributed by `finalize` / `withdraw`. Deposits priced against an understated denominator mint too many LP, letting depositors extract a share of unpriced assets on withdrawal.

```
if ((*v > 0) && (pyth_oracle.contain_oracle_info(*k))) {
    valid_assets.insert(*k, *v);
};

if ((*v > 0) && (pyth_oracle.contain_oracle_info(*k))) {
    if (valid_assets.contains(k)) {
        let amount = valid_assets.get_mut(k);
        *amount = *amount + *v;
    } else {
        valid_assets.insert(*k, *v);
    }
};
```

Recommendation

We would like to check with the team if the scenario has been considered.

Alleviation

[Haedal, 11/20/2025]: We acknowledge the issue, but the observed behavior is expected and does not lead to unfair LP minting. Because all users' deposits and AUM calculations consistently exclude assets without oracle prices, the LP minting process remains fair and economically aligned.

The only temporary imbalance occurs between the last reward collection and the moment a new user deposits: unpriced reward-tokens generated during this interval may cause the new depositor to share in a small amount of previously accrued rewards. However, since our keeper harvests rewards regularly and moves them into the buffer, the quantity of unpriced rewards accumulated between harvests is tightly limited. As a result, any impact is minor and fully acceptable within our system's design parameters.

HVV-04 | Missing Check On `usd_price_age`

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/pyth_oracle.move (vault-v2): 293	● Resolved

Description

The `add_oracle_info()` function allows the oracle manager to update oracle information. This value is used to determine price freshness: a price is considered fresh and valid only if the difference between the current time and the price update time is less than this threshold.

However, there is no input validation on the `usd_price_age` parameter. For example, this value should always be greater than zero. If it is set to zero or another unreasonable value, prices would never be recognized as fresh, preventing updates from being accepted as expected.

Recommendation

It is recommended to add input validation on `usd_price_age` within the `update_price_age()` and `add_oracle_info()` functions.

Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by adding validation on `usd_price_age` in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#).

HVV-05 | Incomplete Logic For `remove_full_range_liquidity_by_percent`

Category	Severity	Location	Status
Coding Issue	● Informational	packages/volatile_vault_v2/sources/market_dlmm.move (vault-v2): 424~425	● Resolved

Description

The function accepts `lp_amount` and `total_lp_amount` but never uses them. The withdrawal percentage passed to `pool::remove_liquidity_by_percent` is hardcoded to 10, making the logic incomplete and the API misleading.

Recommendation

Compute percent from inputs, handle edge cases (zero/overflow, min/max caps), and update tests/docs accordingly.

Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by using input `lp_amount` and `total_lp_amount` in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#).

HVV-07 | Inconsistency Between Code And Comment

Category	Severity	Location	Status
Coding Issue	● Informational	packages/volatile_vault_v2/sources/balance_bag.move (vault-v2): 105	● Resolved

Description

The comment for `withdraw_all` states it will set the tracked balance to 0, but it actually removes the key entirely from the vector map.

`balance_bag.move`:

```
/// Updates the tracked balance amount in the balances map to 0.
...
/// - Removes entire balance of CoinType from the bag
@> /// - Sets tracked balance to 0 in balances map
/// - Returns the removed balance
public(package) fun withdraw_all<CoinType>(bag: &mut BalanceBag): Balance<CoinType>
{
    let (balance, _) = utils::remove_balance_from_bag<CoinType>(&mut bag.bag, 0,
true);
    let ty = type_name::with_defining_ids<CoinType>();
    if (bag.balances.contains(&ty)) {
@>        bag.balances.remove(&ty);
    };
    balance
}
```

Recommendation

Recommend fixing the inconsistency between the code and the comment.

Alleviation

[Haedal, 11/18/2025]: The team heeded the advice and resolved the issue by correcting the comments in commit [5b6b28f4b14de4f807e6efd6d84381ff8311ac9b](#)

HVV-09 | Potential Multiple Market Support In Vault

Category	Severity	Location	Status
Design Issue	● Informational	packages/volatile_vault_v2/sources/vault.move (11/17 - 5b6 b28): 670	● Acknowledged

Description

The current vault implementation implicitly assumes a single market keyed by `DLMM_MARKET_KEY`. Most of the DLMM-specific keeper actions correctly guard with `contains(DLMM_MARKET_KEY)`, but some aggregation logic only aggregates DLMM data conditionally, which will silently undercount and underreport once additional market keys are introduced in the future. For example:

```
public fun calculate_aum<T>(  
    vault: &mut Vault<T>,  
    pyth_oracle: &PythOracle,  
    versioned: &Versioned,  
    clk: &Clock,  
    ctx: &TxContext,  
) {  
    ...  
> if (vault.markets.contains(DLMM_MARKET_KEY)) {  
    let dlmm_market = vault  
        .markets  
        .borrow_mut<vector<u8>, DLMMMarket>(DLMM_MARKET_KEY);  
    let position_amounts = dlmm_market.last_amounts(ctx);  
    let mut idx = 0;  
    while (idx < position_amounts.length()) {  
        let (k, v) = position_amounts.get_entry_by_idx(idx);  
        if ((*v > 0) && (pyth_oracle.contain_oracle_info(*k))) {  
            valid_assets.insert(*k, *v);  
        };  
        idx = idx + 1;  
    };  
};  
    ...  
}
```

The same issue exists in the `get_vault_assets` and `new_withdraw_cert`.

Recommendation

We would like to confirm the purpose of `DLMM_MARKET_KEY` with the client. Could this key be changed in the future? Will the Vault support multiple keys in the future?

I Alleviation

[Haedal, 11/20/2025]: Regarding the concerns about `DLMM_MARKET_KEY`, we confirm that `DLMM_MARKET_KEY` ** is fixed and will not change in the future**. The current vault design intentionally treats DLMM as a single, dedicated market under this constant key.

When introducing additional market types, we will **add new market keys**, such as a planned `CLMM_MARKET_KEY`. Each new market will be integrated as an independent entry in `vault.markets`, rather than reusing or altering the existing DLMM key.

This means future multi-market support will come from **adding new market keys**, and not from modifying `DLMM_MARKET_KEY`. We will also update the aggregation logic accordingly to ensure all market types are properly included in AUM calculations, asset collection, and withdrawal certificate logic.

HVV-10 | Potential Inconsistent Pool Price Deviation Checks Across DLMM Adding Liquidity/Opening Position

Category	Severity	Location	Status
Design Issue	● Informational	packages/volatile_vault_v2/sources/vault.move (vault-v2): 1057	● Resolved

Description

Only `open_dlmm_position` enforces oracle/pool price deviation checks before consuming buffer assets. Other open/adder paths skip this guard, allowing liquidity to be added while price deviates beyond configured thresholds.

```
dlmm_market.check_pool_price_deviation(pool, pyth_oracle, clk);
```

Recommendation

Recommend adding the price-deviation check to `open_position_{spot,curve,bid_ask}` and all `add_*` functions

Alleviation

[Haedal, 11/21/2025]: Fixed

<https://github.com/haedalsd/haevault/pull/8/commits/650021e485f63193a7f5dc0fea1c4871fa6635e3>

[CertiK, 11/24/2025]: Thank you for your reply and the fix. Currently, the non-strategy flow

`new_dlmm_add_liquidity_cert()` → `add_dlmm_liquidity_on_bin()` → `repay_dlmm_add_liquidity()` has not yet added the corresponding checks.

[Haedal, 11/26/2025]: <https://github.com/haedalsd/haevault/pull/8/commits/ae1ef9649e7beee1f61e4bd61ff95f7adf6d8dbf>

We've added the necessary validation to the `new_dlmm_add_liquidity_cert` method.

Since the other two methods are always invoked together with this one, their checks are now implicitly covered, so we are not adding separate validations for them.

[CertiK, 11/26/2025]: The team heeded the advice and resolved the issue by adding a price check in

`new_dlmm_add_liquidity_cert` in commit [ae1ef9649e7beee1f61e4bd61ff95f7adf6d8dbf](https://github.com/haedalsd/haevault/pull/8/commits/ae1ef9649e7beee1f61e4bd61ff95f7adf6d8dbf).

HVV-12 Remove On-Chain Dependency On Pyth

`update_single_price_feed` To Prevent Unintended Abort

Category	Severity	Location	Status
Design Issue, Code Optimization	● Informational	packages/volatile_vault_v2/sources/pyth_oracle.move e (11/17 - 5b6b28): 539~545	● Resolved

Description

The module `pyth_oracle.move` hard-codes a call to `pyth::update_single_price_feed` inside on-chain logic. Pyth's official guidance prohibits this because Pyth upgrades change the package address, and only the latest package is allowed to accept updates. This creates a potential abort risk: after a Pyth upgrade, the on-chain call path can fail, breaking price updates and any dependent flows.

Official Warning (<https://docs.pyth.network/price-feeds/core/use-real-time-data/pull-integration/sui#write-contract-code>)

```
Your Sui Move module should NOT have a hard-coded call to
pyth::update_single_price_feed. In other words, a contract should never call the Sui
Pyth pyth::update_single_price_feed entry point. Instead, it should be called
directly from client code (e.g., Typescript or Rust).
```

```
When Sui contracts are upgraded, the address changes, which makes the old address no
longer valid. If your module has a hard-coded call to pyth::update_single_price_feed
living at a fixed call-site, it may eventually get bricked due to how Pyth upgrades
are implemented. (Pyth only allow users to interact with the most recent package
version for security reasons).
```

```
Therefore, you should build a Sui programmable transaction that first updates the
price by calling pyth::update_single_price_feed at the latest call-site from the
client-side and then call a function in your contract that invokes pyth::get_price
on the PriceInfoObject to get the recently updated price. You can use SuiPythClient
to build such transactions and handle all the complexity of updating the price
feeds.
```

```
Consult Fetch Price Updates for more information on how to fetch the
pyth_price_update.
```

`pyth_oracle.move`:

```

public fun update_price<T>(
    pyth_oracle: &mut PythOracle,
    pyth_state: &State,
    price_updates: HotPotatoVector<PriceInfo>,
    price_info_obj: &mut PriceInfoObject,
    clk: &Clock,
    versioned: &Versioned,
    ctx: &mut TxContext,
): HotPotatoVector<PriceInfo> {
    versioned.check_version();

    let fee =
pyth_oracle.split_fee(pyth_state.get_base_update_fee()).into_coin(ctx);
>@ let h = pyth::update_single_price_feed(
    pyth_state,
    price_updates,
    price_info_obj,
    fee,
    clk,
);
    ...
}

```

Recommendation

We would like to check with the team if the scenario has been considered.

Alleviation

[Haedal, 11/24/2025]: The team added a new `update_price_into_pyth_oracle()` function in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#).

[CertiK, 11/24/2025]: The new `update_price_into_pyth_oracle()` function doesn't require users to fetch and update the latest price data to the Pyth Oracle since no a hotpotato object input. As a result, users can directly use an older price, as long as it remains within the `max_age`, without having to update the Oracle in the same seconds.

Even with the older methods, the user may not be using the latest price feed as long as it is acceptable with the age threshold.

We would like to check with the team if the design is intended.

[Haedal, 11/26/2025]: Thanks for the question.

This behavior is acceptable and intended. We've reviewed Pyth's `PriceInfo.arrival_time` field and confirmed that even when `update_single_price_feed` is called, this timestamp is not always updated immediately. Because of this, relying solely on `arrival_time` is not sufficient to guarantee freshness.

Instead, we enforce correctness through the `price_age` constraint, which ensures that the price used in the transaction is always within the allowed age window. As long as the price satisfies this age requirement, it is considered safe and valid,

even if the oracle itself was not updated in the same second.

HVV-13 | Debug Import Left In Production Code

Category	Severity	Location	Status
Coding Style	● Informational	packages/volatile_vault_v2/sources/market_dlmm.move (11/17 - 5b6b28): 818~819	● Resolved

Description

The `market_dlmm.move` file includes an import of "std::debug::print", which is intended solely for local development and testing.

```
public fun check_pool_price_deviation<CoinTypeA, CoinTypeB>(
    market: &DLMMMarket,
    pool: &Pool<CoinTypeA, CoinTypeB>,
    pyth_oracle: &PythOracle,
    clk: &Clock,
) {
    let base_price = pyth_oracle.get_price<CoinTypeA>(clk);
    let quote_price = pyth_oracle.get_price<CoinTypeB>(clk);
    let (pool_oracle_price, _) = calculate_prices(&base_price, &quote_price);
    let pool_current_price = get_current_pool_price(pool);
    @> std::debug::print(&pool_oracle_price);
    @> std::debug::print(&pool_current_price);
}
```

Recommendation

It's recommended to remove debugging code.

Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by removing debugging code in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#)

HVV-15 | Oracle Price Cache Not Cleared When Oracle Info Is Removed

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/pyth_oracle.move (vault-v2): 386~387	● Resolved

Description

The PythOracle stores both oracle configuration (oracle_infos) and a per-type cached price (prices). When oracle info is removed via `remove_oracle_info<T>`, only the config entry is deleted; the cached price entry is left untouched, which raises concerns about the illegal price inside the same block, which may bypass the **same block** validation in `get_price / get_price_by_type`.

Recommendation

Recommend removing the price as well when removing the oracle.

Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by removing the price when removing the oracle in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#)

HVV-16 Incorrect Price Decimals Calculation In `calculate_oracle_prices()`

Category	Severity	Location	Status
Incorrect Calculation	● Informational	packages/volatile_vault_v2/sources/pyth_oracle.move (vault-v2): 878~898	● Resolved

Description

The `calculate_prices_from_base_quote()` function takes the tokens' respective `coin_decimals` into account when computing `base_to_quote_factor` and `quote_to_base_factor`. When the two tokens' `coin_decimals` differ significantly, one of the calculated prices may become very large, while the other will have low precision. When the number becomes excessively large, it will overflow because the return value is a `u64`. And when the precision is too low, the price may not be represented correctly. Although most coins on Sui currently use 6, 8, or 9 decimals—resulting in at most/least 13/7 price decimals—the potential overflow risk and the price precision issue caused by a large difference in decimals should not be overlooked.

`pyth_oracle.move`

```
878 fun calculate_prices_from_base_quote(  
879     base_price: u64,  
880     quote_price: u64,  
881     base_decimals: u8,  
882     quote_decimals: u8,  
883 ): (u64, u64) {  
884     // Calculate scaling factor for base/quote  
885     let base_to_quote_factor = u64::pow(  
886         10,  
887         @> price_multiplier_decimal!() + quote_decimals - base_decimals,  
888     );  
889     let quote_to_base_factor = u64::pow(  
890         10,  
891         @> price_multiplier_decimal!() + base_decimals - quote_decimals,  
892     );  
893     // Calculate prices  
894     @> let primitive_base_price = mul_div_floor(base_price, base_to_quote_factor,  
895     , quote_price);  
895     @> let primitive_quote_price = mul_div_floor(quote_price,  
896     quote_to_base_factor, base_price);  
896  
897     (primitive_base_price, primitive_quote_price)  
898 }
```

Scenario

Assume base has 3 decimals (an BTC pegged coin), quote has 9 decimals (e.g., SUI) -> USD prices from Pyth (already scaled by 10^{10}): $\text{base_price} = 100000.00 \rightarrow 1,000,000,000,000,000$ $\text{quote_price} = 3.50 \rightarrow 35,000,000,000$

Factors: $\text{base_to_quote_factor} = 10^{(10 + 9 - 3)} = 10^{16}$ $\text{quote_to_base_factor} = 10^{(10 + 3 - 9)} = 10^4$

Primitive prices: $\text{primitive_base_price} = \text{base_price} * 10^{16} // \text{quote_price} = 285,714,285,714,285,690,880$ (this equals $\text{quote}/\text{base} \times 10^{(10 + 9 - 3)}$) $\text{primitive_quote_price} = \text{quote_price} * 10^4 // \text{base_price} = 0$ (this equals $\text{base}/\text{quote} \times 10^{(10 + 3 - 9)}$)

It can be seen that `primitive_base_price` exceeds the upper limit of `u64` due to its large value, while `primitive_quote_price` has insufficient precision, resulting in a price of 0.

Recommendation

We recommend fixing the precision to `price_multiplier_decimal()`.

Alleviation

[Haedal, 11/21/2025]: <https://github.com/haedallsd/haevault/pull/8/commits/7ed286eaf442255cea0107ebef1ed653c016d96f>

[CertiK, 11/21/2025]: Thank you for your reply and the fix. The current fix extends the returned price to `u128` to prevent overflow, but it does not address the issue shown in the PoC where the price becomes 0 due to excessively low precision.

[Haedal, 11/26/2025]: <https://github.com/haedallsd/haevault/pull/8/commits/ae1ef9649e7beee1f61e4bd61ff95f7adf6d8dbf>

We use a dedicated `relative_price_multiplier_decimal` to apply a larger scaling factor when computing relative token prices. This ensures higher precision during relative price calculations and avoids rounding issues in downstream logic.

[CertiK, 11/26/2025]: We want to remind the team that `pyth_oracle::calculate_oracle_prices` still uses the old `calculate_prices_from_base_quote`. This should be fine for backward compatibility, but we also recommend having a v2 version with `calculate_prices_from_base_quote_v2`.

[Haedal, 12/01/2025]: The `calculate_oracle_prices` method is actually not used and may be removed in the future.

HVV-17 | `flash_loan` Allows Identical `CoinTypeA` And `CoinTypeB`

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/vault.move (vault-v2): 1790	● Resolved

Description

The function does not enforce `CoinTypeA != CoinTypeB`. When both generics are the same type, price/repay logic assumes cross-asset conversion but can perform a same-asset flash loan, making the repay-type check trivial and producing misleading behavior.

```
public fun flash_loan<CoinTypeA, CoinTypeB, LPCoin>(
    vault: &mut Vault<LPCoin>,
    pyth_oracle: &PythOracle,
    loan_amount: u64,
    config: &GlobalConfig,
    versioned: &Versioned,
    clk: &Clock,
    ctx: &mut TxContext,
): (Coin<CoinTypeA>, FlashLoanCert) {
    ...
    let swap_slippage_a = config::get_swap_slippage<CoinTypeA>(config);
    let swap_slippage_b = config::get_swap_slippage<CoinTypeB>(config);
    let swap_slippage = (swap_slippage_a + swap_slippage_b) / 2;
    let repay_amount = full_math_u64::mul_div_ceil(
        repay_amount,
        (slippage_denominator!() - swap_slippage),
        slippage_denominator!(),
    );
    let (loan_type, repay_type) = (
        type_name::with_defining_ids<CoinTypeA>(),
        type_name::with_defining_ids<CoinTypeB>(),
    );
    ...
}
```

Similarly, the `create_vault` also does not enforce `CoinTypeA != CoinTypeB`.

Recommendation

Ensure `CoinTypeA != CoinTypeB`.

Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by adding corresponding check in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#)

HVV-18 | DLMM Fee/Reward Helpers Lack WithdrawCert–Vault Binding And State Checks

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/vault.move (vault-v2): 920	● Resolved

Description

The following functions operate on the vault's DLMM state and finally mutate the WithdrawCert:

- vault::collect_dlmm_fee_on_withdraw
- vault::collect_dlmm_reward_on_withdraw

However, both functions are missing:

- binding check: that ensures the given vault is the same as the one init the WithdrawCert
- state check: that the WithdrawCert is still pending
- position check: that the given position may not address to the WithdrawCert

Recommendation

Recommend adding aforementioned checks to avoid unexpected execution.

Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by adding corresponding checks in commit [9b220591ce962753e0dedd4defb897f66fc839a8](#)

HVV-19 | Pyth Oracle Integration Ignores Confidence `conf` On Prices

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/pyth_oracle.move (vault-v2): 253	● Resolved

Description

The Pyth Sui contracts expose a Price struct that includes a confidence interval `conf` intended to represent the uncertainty around the reported price:

```
/// A price with a degree of uncertainty, represented as a price +- a confidence interval.
struct Price has copy, drop, store {
  price: I64,
  /// Confidence interval around the price
  conf: u64,
  /// The exponent
  expo: I64,
  /// Unix timestamp of when this price was computed
  timestamp: u64,
}
...
public fun get_conf(price: &Price): u64 {
  price.conf
}
```

In the project's `pyth_oracle` module, `pyth_price_from_oracle_info` converts Pyth's Price into a u64, but only checks age, feed id, sign, and non-zero value. It uses `price::get_price` and `price::get_expo`, and never reads or constrains `price.conf`.

Recommendation

Recommend adding validation on Oracle confidence as well to avoid unexpected prices, since the project is price sensitive.

Alleviation

[Haedal, 11/24/2025]: The team heeded the advice and resolved the issue by adding confidence check in commit [9b220591ce962753e0dedd4defb897f66fc839a8](https://github.com/haedal/haedal-vault-v2/commit/9b220591ce962753e0dedd4defb897f66fc839a8).

HVV-20 | Potential Incorrect Slippage Application For Repay Amount

Category	Severity	Location	Status
Design Issue	● Informational	packages/volatile_vault_v2/sources/vault.move (11/17 - 5b6b28): 1886~1893	● Resolved

Description

The flash loan repayment applies the configured swap slippage in the wrong direction. Instead of increasing the repayment to account for slippage and execution risk ("plus slippage" per the comment), the current code reduces the required repayment by multiplying by `(1 - slippage)`. This allows underpayment relative to the oracle fair price.

```
let swap_slippage_a = config::get_swap_slippage<CoinTypeA>(config);
let swap_slippage_b = config::get_swap_slippage<CoinTypeB>(config);
let swap_slippage = (swap_slippage_a + swap_slippage_b) / 2;
let repay_amount = full_math_u64::mul_div_ceil(
    repay_amount,
    (slippage_denominator!() - swap_slippage),
    slippage_denominator!(),
);
```

Recommendation

Since this operation can only be called by a privileged operator, we recommend the team double-check the intended design.

Alleviation

[Haedal, 10/10/2025]: This behavior is intentional.

The design allows the repayment amount to be slightly **lower** than the value calculated from oracle price, applying slippage as a **discount**, not an addition.

The parameter `swap_slippage` represents a tolerance for reduced repayment, not extra cost.

HVV-22 | Potential Issue Of Unfair Reward Distribution

Category	Severity	Location	Status
Design Issue	● Informational	packages/volatile_vault_v2/sources/vault.move (vault-v2): 552	● Resolved

Description

In `deposit()`, users can deposit any token whose price has been updated at the current timestamp in the `pyth_oracle`. As a result, users may deposit tokens that are not the vault's `coin_type_a` or `coin_type_b`.

However, allowing deposits of tokens other than `coin_type_a` and `coin_type_b` introduces unfairness in reward distribution. For deposits of non-`coin_type_a` / `coin_type_b` tokens, the keeper manager can not stake them into the DLMM to earn fees and rewards. Therefore, such tokens do not participate in the liquidity-staking process performed by the vault's keeper manager, and thus should not be entitled to receive the corresponding fees and rewards. Likewise, because these tokens are not involved in liquidity provision, they should not bear the economic loss associated with such staking.

Recommendation

We would like to confirm with the official team whether this behavior is intentional.

Alleviation

[Haedal, 11/26/2025]: <https://github.com/haedallsd/haevault/pull/8/commits/ae1ef9649e7beee1f61e4bd61ff95f7adf6d8dbf>

[CertiK, 11/26/2025]: The team restricted the deposit function so it only accepts coin A and coin B, in commit [ae1ef9649e7beee1f61e4bd61ff95f7adf6d8dbf](https://github.com/haedallsd/haevault/pull/8/commits/ae1ef9649e7beee1f61e4bd61ff95f7adf6d8dbf).

HVV-24 | Version Control Did Not Emit Event

Category	Severity	Location	Status
Logical Issue	● Informational	packages/volatile_vault_v2/sources/config.move (11/21 - 9b2205): 151~157, 166~179	● Resolved

Description

The contract uses the two privileged functions `emergency_pause()` and `emergency_unpause()` for pausing contract functions. However, neither of these functions emits the corresponding event.

Recommendation

We recommend adding events for all privileged functions.

Alleviation

[Haedal, 11/26/2025]: The team heeded the advice and resolved the issue by adding corresponding events in commit [6d94df586ccad1cad3f2f6bd7190962eab4f4873](#).

HVV-27 | Concerns About `update_price` And `get_price`

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/pyth_oracle.move (vault-v2): 527	● Resolved

Description

The `update_price` function is publicly being used by anyone to update the Pyth oracle before any `get_price` related operations.

However, `update_price<T>` deducts the update fee from the protocol's `pyth_oracle.update_price_fee` before checking if the token T is actually registered/supported; as a result, an arbitrary party may repeatedly call `update_price` for unregistered tokens to use up all `update_price_fee`.

Recommendation

Recommend adding validation to avoid unexpected token type consuming the update fee.

Alleviation

[Haedal, 11/26/2025]: The team heeded the advice and resolved the issue by adding corresponding events in commit [6d94df586ccad1cad3f2f6bd7190962eab4f4873](#).

HVV-28 Compilation Error In Unit Testing

Category	Severity	Location	Status
Coding Issue	● Informational		● Resolved

Description

There is a compilation error in the test file `market_dllmm_tests.move`.

```

error[E04007]: incompatible types
  └─ ./tests/market_dlmm_tests.move:1006:9
1006 |     let (cert, balance_a, balance_b) = market_dlmm::close_position_v2<USDC,
SUI>(
    |
    |          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |          |
    |          Invalid value for binding
    |          Expected expression list of length 3: '(_, _, _)')
  └─ ./sources/market_dlmm.move:663:4
663 | ): (DLMMPositionCloseCertV2, Balance<CoinTypeA>, Balance<CoinTypeB>,
Balance<CoinTypeA>, Balance<CoinTypeB>) {
    |
    |----- Given expression list of length 5:
    |'(volatile_vault_v2::market_dlmm::DLMMPositionCloseCertV2,
    |sui::balance::Balance<volatile_vault_v2::market_dlmm_tests::USDC>,
    |sui::balance::Balance<volatile_vault_v2::market_dlmm_tests::SUI>,
    |sui::balance::Balance<volatile_vault_v2::market_dlmm_tests::USDC>,
    |sui::balance::Balance<volatile_vault_v2::market_dlmm_tests::SUI>)'

```

Recommendation

Recommend fixing the compilation errors.

■ Alleviation

[Haedal, 12/01/2025]:

<https://github.com/haedallsd/haevault/pull/8/commits/0cd6eb12225ebef856e69a27e43e0da1bb216482>

[CertiK, 12/10/2025]: The team heeded the advice and resolved the issue in commit

0cd6eb1225ebef856e69a27e43e0da1bb216482.

HVV-29 | Event Amounts Exclude Fee Balances Credited To The Vault

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/vault.move (12/09 - 467e91): 2632~2638	● Resolved

Description

`DimmClosePositionEvent.total_amount_a/total_amount_b` are emitted using `balance_a.value()` and `balance_b.value()` before `fee_a` and `fee_b` are merged into these balances and joined to `buffer_assets`. As a result, off-chain consumers relying on the close event will underreport the total amounts actually credited to the vault

Recommendation

We would like to check with the team if this is the intended design.

Alleviation

[Haedal, 12/11/2025]: The team heeded the advice and resolved the issue by updating log values in commit [f9881cdd89b22098509c2d34533688df76d4ee9b](#).

HVV-30 | Keeper Operations Not Pausable

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/vault.move (12/09 - 467 e91): 1590	● Acknowledged

Description

The keeper functions does not check any vault-level pause flag before proceeding. While

`deposit/withdraw/calculate_aum` paths enforce `action_status` via `is_allow_*` checks, keeper operations (including `increase_clmm_liquidity`) only verify roles and last-tx constraints.

This makes it impossible to pause keeper operations via `update_active_status`, contradicting the intended "pausable operations" design. Add an allow/deny check (e.g., `is_allow_keeper_operation`) analogous to deposit/withdraw, and assert it here to allow admins to pause keeper actions when required.

Recommendation

We would like to check with the team if this is intended.

Alleviation

[Haedal, 12/10/2025]: We can pause keeper execution through ACL management by revoking or restricting the keeper's assigned permissions. Therefore, keeper operations can still be effectively paused via access control, even though they do not rely on the vault-level pause flag.

HVV-34 | `ClmmAddLiquidityEvent` Logs Allowed Amounts Instead Of Actual Used Amounts

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/vault.move (12/09 - 467e91): 1631~1640	● Resolved

Description

The event emits `total_amount_a/total_amount_b` equal to the allowed input amounts (`amount_a/amount_b`), not the amounts actually consumed by CLMM.

```
1631 vault.buffer_assets.join(balance_a);
1632 vault.buffer_assets.join(balance_b);
1633 emit(ClmmAddLiquidityEvent {
1634     vault_id: object::id(vault),
1635     position_id,
1636     pool_id: object::id(clmm_pool),
1637     current_sqrt_price: clmm_pool.current_sqrt_price(),
1638     total_amount_a: amount_a,
1639     total_amount_b: amount_b,
1640 });
```

Recommendation

Recommend emitting actual value for adding liquidity.

Alleviation

[Haedal, 12/11/2025]: The team heeded the advice and resolved the issue by updating log values in commit [f9881cdd89b22098509c2d34533688df76d4ee9b](#).

HVV-38 Potential DoS At Minimum Representable Price For `price == 1`

Category	Severity	Location	Status
Volatile Code	● Informational	packages/volatile_vault_v2/sources/utils.move (12/18 - 8169f2): 119	● Resolved

Description

For `price < ONE_Q64`, the function computes `inv = (ONE_Q64 * ONE_Q64) / price` in `u256` and then asserts `inv <= u128::max_value!()` before casting back to `u128`.

When `price == 1` (the minimum non-zero Q64.64 value, representing 2^{-64}), `inv` becomes exactly `2128`, which is one greater than `u128::max_value!()` ($2^{128} - 1$). This triggers `IdOutOfRangeError` and aborts.

Recommendation

Recommend handling corner cases and perform corresponding test if not intended to avoid extreme values.

Alleviation

[Haedal, 12/22/2025]: The team heeded the advice and resolved the issue by adding additional logic to handle "price == 1" in commit [8a11a17dfc3f667047c315a0497fdd247c576e66](#).

OPTIMIZATIONS | HAEDAL - VOLATILE VAULT V2

ID	Title	Category	Severity	Status
HVV-06	Variable Naming	Coding Style	Optimization	● Resolved
HVV-21	The Complex Withdraw And Deposit Process Consumes Significant Gas	Logical Issue	Optimization	● Acknowledged

HVV-06 | Variable Naming

Category	Severity	Location	Status
Coding Style	● Optimization	packages/volatile_vault_v2/sources/vault.move (11/17 - 5b6b28): 3 97; packages/volatile_vault_v2/sources/acl.move (vault-v2): 161~1 67	● Resolved

Description

A variable is named `address`, the same as the imported module (`sui::address`). While this does not cause semantic shadowing for qualified calls (`address::...`), the name collision creates ambiguity.

- Users may misinterpret usages as shadowing, increasing cognitive load during audits, debugging, and integrations.
- Future refactors that introduce/modify a use alias (e.g., alias `sui::address` to `address`) or adjust imports can trigger name conflicts, compilation errors, or broken references.

Recommendation

It is recommended to use a different variable name instead of `address`.

Alleviation

[Haedal, 11/18/2025]: The team heeded the advice and resolved the issue by updating the variable name in commit [5b6b28f4b14de4f807e6efd6d84381ff8311ac9b](#).

[CertiK, 11/20/2025]: We found another place with the same issue:

```
vault.move
```

```
397 let type_name = type_name::with_defining_ids<LPCoin>();
```

We recommend that the team use a different name.

[CertiK, 11/26/2025]: The team heeded the advice and resolved the issue by updating the variable name in commit [ae1ef9649e7beee1f61e4bd61ff95f7adf6d8dbf](#).

HVV-21 | The Complex Withdraw And Deposit Process Consumes Significant Gas

Category	Severity	Location	Status
Logical Issue	● Optimization	packages/volatile_vault_v2/sources/vault.move (vault-v2): 55 2, 617, 672, 697, 747, 782, 807, 846, 920, 976	● Acknowledged

Description

Currently, both the withdraw and deposit processes involve calling multiple functions within a single PTB, resulting in high complexity and potentially excessive gas consumption. In severe cases, this could even introduce a DoS risk.

Deposit

Execution flow: `pyth_oracle::update_price()` → `calculate_dlmm_position_amounts()` → `calculate_aum()` → `deposit()`

1. `pyth_oracle::update_price()` must update the prices of all tokens required by `calculate_aum()`, meaning all tokens in the vault (including position pair tokens and reward tokens). This ensures that the price update timestamp in the PTB matches the current time so that prices remain valid. The number of calls depends on how many token types the vault contains.
2. `calculate_dlmm_position_amounts()` must be called for every position in dlmm market. Thus, the number of calls increases with the number of positions.
3. Inside `calculate_dlmm_position_amounts()`, the `last_amounts()` function performs nested loops over `VecMap`, which leads to high complexity.

These three factors collectively cause the deposit process to consume substantial gas.

Withdraw

Execution flow: `new_withdraw_cert()` → `withdraw_dlmm()` → `collect_dlmm_reward_on_withdraw()` → `collect_dlmm_fee_on_withdraw()` → `finalize_buffer_assets()` → `withdraw_asset()` → `destroy_withdraw_cert()`

1. The withdraw process itself contains a long execution chain, resulting in high gas consumption.
2. `withdraw_dlmm()` must be executed once for each position and internally calls Cetus DLMM's complex `remove_full_range_liquidity_by_percent()`.
3. `collect_dlmm_reward_on_withdraw()` must be called once per position per reward type — i.e., `number_of_positions × number_of_reward_tokens`.

4. `collect_dlm_fee_on_withdraw()` also needs to be called once per position.
5. In `finalize_buffer_assets()`, iterating over `withdraw_cert.rewards` and `vault.buffer_assets` involves nested loops on two VecMaps, leading to high time complexity.
6. `withdraw_asset()` must be called once for every token type in the vault.

These six factors result in significant gas consumption during the withdraw process.

Recommendation

We recommend that the development team review the complexity and gas usage of both the deposit and withdraw processes.

Alleviation

[Haedal, 11/26/2025]:

In our case, the number of `MoveCall` operations is not the primary source of high gas consumption. The main driver is the large number of bins within the underlying DLMM position, which significantly increases gas usage. Therefore, the only practical way to control gas costs is to manage and limit the number of bins at the strategy level.

APPENDIX | HAEDAL - VOLATILE VAULT V2

Audit Scope

haedallsd/haevault

- packages/volatile_vault_v2/sources/vault.move
- packages/volatile_vault_v2/sources/vault.move
- packages/volatile_vault_v2/sources/vault.move
- packages/volatile_vault_v2/sources/acl.move
- packages/volatile_vault_v2/sources/balance_bag.move
- packages/volatile_vault_v2/sources/config.move
- packages/volatile_vault_v2/sources/market_dlmm.move
- packages/volatile_vault_v2/sources/pyth_oracle.move
- packages/volatile_vault_v2/sources/market_dlmm.move
- packages/volatile_vault_v2/sources/pyth_oracle.move
- packages/volatile_vault_v2/sources/config.move
- packages/volatile_vault_v2/sources/vault.move
- packages/volatile_vault_v2/sources/market_clmm.move
- packages/volatile_vault_v2/sources/market_clmm.move
- packages/volatile_vault_v2/sources/utills.move
- packages/volatile_vault_v2/sources/admin_cap.move
- packages/volatile_vault_v2/sources/utills.move
- packages/volatile_vault_v2/sources/versioned.move
- packages/volatile_vault_v2/sources/acl.move

haedalsd/haevault

- packages/volatile_vault_v2/sources/admin_cap.move
- packages/volatile_vault_v2/sources/balance_bag.move
- packages/volatile_vault_v2/sources/config.move
- packages/volatile_vault_v2/sources/utills.move
- packages/volatile_vault_v2/sources/versioned.move
- packages/volatile_vault_v2/sources/acl.move
- packages/volatile_vault_v2/sources/admin_cap.move
- packages/volatile_vault_v2/sources/balance_bag.move
- packages/volatile_vault_v2/sources/market_dlmm.move
- packages/volatile_vault_v2/sources/pyth_oracle.move
- packages/volatile_vault_v2/sources/utills.move
- packages/volatile_vault_v2/sources/versioned.move
- packages/volatile_vault_v2/sources/acl.move
- packages/volatile_vault_v2/sources/admin_cap.move
- packages/volatile_vault_v2/sources/balance_bag.move
- packages/volatile_vault_v2/sources/config.move
- packages/volatile_vault_v2/sources/market_dlmm.move
- packages/volatile_vault_v2/sources/pyth_oracle.move
- packages/volatile_vault_v2/sources/utills.move
- packages/volatile_vault_v2/sources/vault.move
- packages/volatile_vault_v2/sources/versioned.move
- packages/volatile_vault_v2/sources/acl.move

haedalsd/haevault

- packages/volatile_vault_v2/sources/admin_cap.move
- packages/volatile_vault_v2/sources/balance_bag.move
- packages/volatile_vault_v2/sources/config.move
- packages/volatile_vault_v2/sources/market_dlmm.move
- packages/volatile_vault_v2/sources/pyth_oracle.move
- packages/volatile_vault_v2/sources/utils.move
- packages/volatile_vault_v2/sources/vault.move
- packages/volatile_vault_v2/sources/versioned.move
- packages/volatile_vault_v2/sources/acl.move
- packages/volatile_vault_v2/sources/admin_cap.move
- packages/volatile_vault_v2/sources/balance_bag.move
- packages/volatile_vault_v2/sources/config.move
- packages/volatile_vault_v2/sources/market_clmm.move
- packages/volatile_vault_v2/sources/market_dlmm.move
- packages/volatile_vault_v2/sources/pyth_oracle.move
- packages/volatile_vault_v2/sources/utils.move
- packages/volatile_vault_v2/sources/vault.move
- packages/volatile_vault_v2/sources/versioned.move
- packages/volatile_vault_v2/sources/market_dlmm.move
- packages/volatile_vault_v2/sources/vault.move
- packages/volatile_vault_v2/sources/market_clmm.move
- packages/volatile_vault_v2/sources/market_dlmm.move

haedallsd/haevault packages/volatile_vault_v2/sources/utils.move packages/volatile_vault_v2/sources/vault.move

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Incorrect Calculation	Incorrect Calculation findings are about issues in numeric computation such as rounding errors, overflows, out-of-bounds and any computation that is not intended.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

